

Homework 6 Solutions

```
In [1]: import numpy as np
import sklearn as sk
import scipy
import matplotlib
from IPython.display import Image
from scipy.stats import pearsonr
import scipy.stats
from sklearn.linear_model import LassoCV
from sklearn import metrics
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error as mse
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.tree import DecisionTreeRegressor
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_curve
```

Problem 1

```
In [2]: def get_data(n):
x = np.random.uniform(size=n)
y = x**2-x/2.0 + np.random.normal(size=n,scale=0.1)
idx = np.argsort(x)
return(x[idx].reshape((-1,1)),y[idx].reshape((-1,1)))

def true_mean(x):
return(x**2-x/2.0)
```

Part a

```
In [3]: x,y = get_data(50)
plt.scatter(x,y)
plt.plot(x, true_mean(x),c='purple')
fit = DecisionTreeRegressor(max_depth=1).fit(x,y)
plt.plot(x, fit.predict(x), c='r')
```

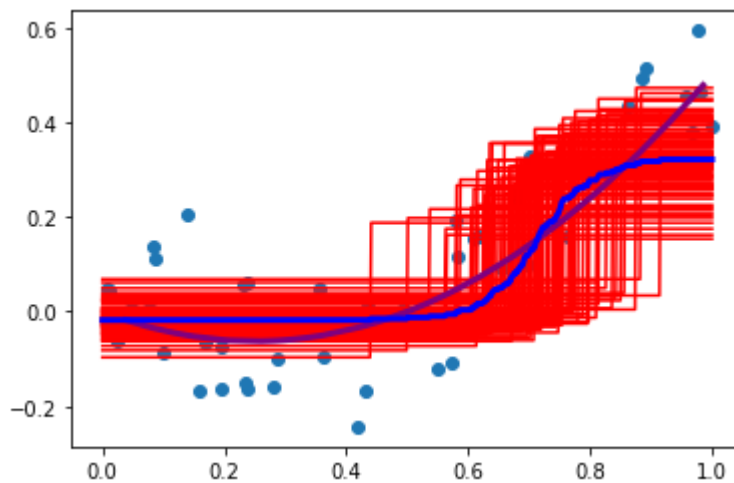
```
Out[3]: [<matplotlib.lines.Line2D at 0x7f3ee5a3ba90>]
```



Part b

```
In [4]: x,y = get_data(50)
xgrid = np.linspace(0,1,500).reshape((-1,1))
preds = np.zeros((100,500))
plt.scatter(x,y)
for i in range(100):
    x,y = get_data(50)
    fit = DecisionTreeRegressor(max_depth=1).fit(x,y)
    plt.plot(xgrid, fit.predict(xgrid), c='r')
    preds[i,:] = fit.predict(xgrid)
plt.plot(x, true_mean(x),c='purple',linewidth=3)
plt.plot(xgrid, np.mean(preds,axis=0), c='b',linewidth=3)
```

Out[4]: [<matplotlib.lines.Line2D at 0x7f3ee5845640>]



The average is much smoother than what we would get from a simple tree, and approximates the conditional mean better. It's still not a great fit though, we see quite a bit of bias in places.

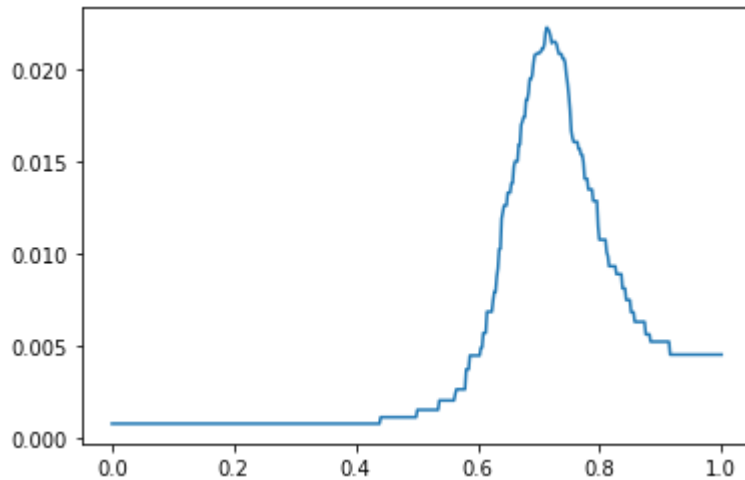
```
In [5]: #Bias
plt.plot(xgrid, np.mean(preds,axis=0).reshape((-1,)) - true_mean(xgrid).resha
```

Out[5]: [<matplotlib.lines.Line2D at 0x7f3ee57c3400>]



```
In [6]: #Variance
plt.plot(xgrid, np.var(preds,axis=0).reshape((-1,)))
```

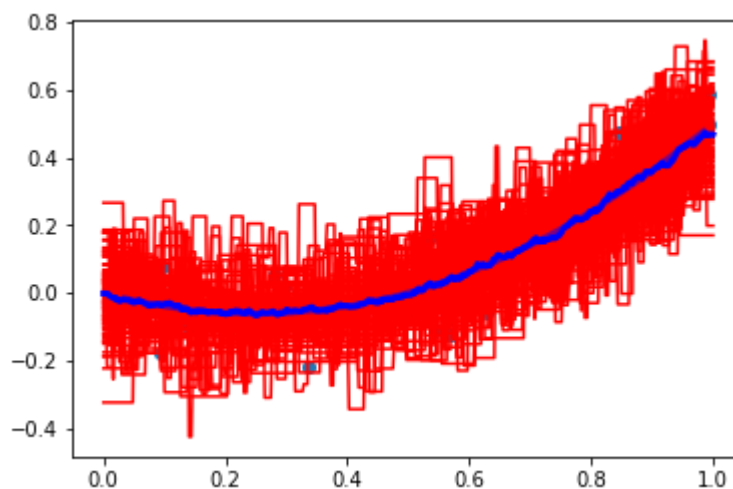
```
Out[6]: [<matplotlib.lines.Line2D at 0x7f3ee5793490>]
```



Part c

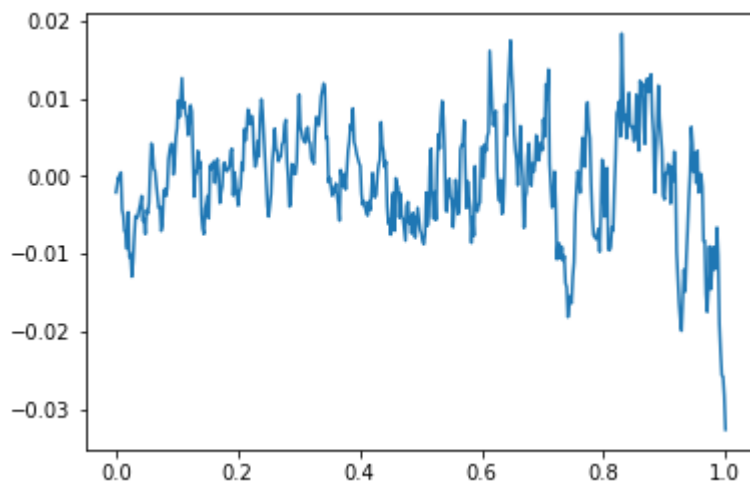
```
In [7]: x,y = get_data(50)
xgrid = np.linspace(0,1,500).reshape((-1,1))
preds = np.zeros((100,500))
plt.scatter(x,y)
for i in range(100):
    x,y = get_data(50)
    fit = DecisionTreeRegressor(max_depth=5).fit(x,y)
    plt.plot(xgrid, fit.predict(xgrid), c='r')
    preds[i,:] = fit.predict(xgrid)
plt.plot(x, true_mean(x),c='purple',linewidth=3)
plt.plot(xgrid, np.mean(preds,axis=0), c='b',linewidth=3)
```

```
Out[7]: [<matplotlib.lines.Line2D at 0x7f3ee566a790>]
```



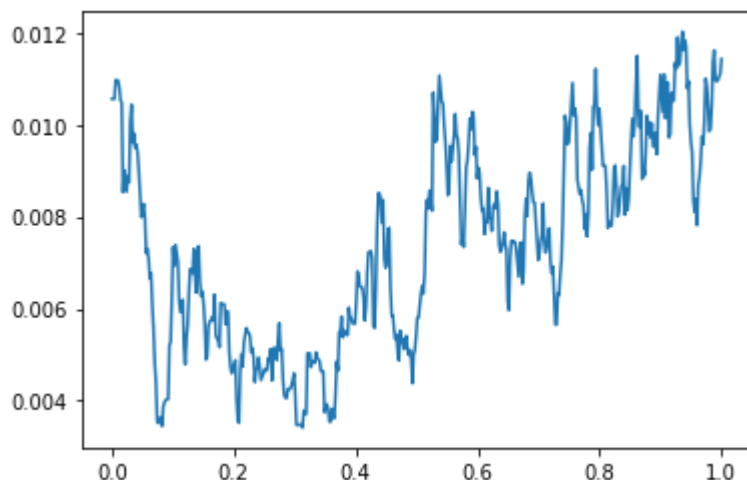
```
In [8]: plt.plot(xgrid, np.mean(preds,axis=0).reshape((-1,)) - true_mean(xgrid).resha
```

Out[8]: [matplotlib.lines.Line2D at 0x7f3ee58022e0]



```
In [9]: plt.plot(xgrid, np.var(preds,axis=0).reshape((-1,)))
```

Out[9]: [matplotlib.lines.Line2D at 0x7f3ee55bb880]



The average of the trees is a much better fit to the conditional mean function. The bias is greatly reduced. The variance is much more evenly spread, though larger on average. This makes sense, since we are using a more variable estimator.

Question 2

pre-processing

```
In [10]: marketing = pd.read_csv("marketing.csv")

# convert the strings to numbers in X
marketing_encoded = marketing.transform(lambda x: LabelEncoder().fit_transform(x))
#Just force y to have yes be 1 (even though, by chance, LabelEncoder does that)
marketing_encoded['y'] = (marketing['y']=='yes').astype('int')
marketing_encoded.head()
```

	age	job	marital	education	default	balance	housing	loan	y
0	58	4	1	2	0	2143	1	0	0
1	44	9	2	1	0	29	1	0	0
2	33	2	1	1	0	2	1	1	0
3	47	1	1	3	0	1506	1	0	0
4	33	11	2	3	0	1	0	0	0

```
In [11]: #Now we can split our sample.
X_train, X_test, y_train, y_test = train_test_split(marketing_encoded.drop(co
                                                    test_size = 0.33,
                                                    random_state = 1)
```

a)

```
In [12]: rf = RandomForestClassifier(n_estimators = 500, oob_score = True)
rf.fit(X_train, y_train)
RandomForestClassifier(bootstrap=True, class_weight=None, criterion='gini',
                        max_depth=None, max_features='auto', max_leaf_nodes=None,
                        min_impurity_decrease=0.0, min_impurity_split=None,
                        min_samples_leaf=1, min_samples_split=2,
                        min_weight_fraction_leaf=0.0, n_estimators=500, n_jobs=None,
                        oob_score=True, random_state=None, verbose=0, warm_start=False)
```

```
Out[12]: RandomForestClassifier(n_estimators=500, oob_score=True)
```

a1)

```
In [13]: y_test_pred_rf = rf.predict(X_test)
rf_misclass_rate = np.mean(y_test_pred_rf != y_test)
rf_misclass_rate

misclass_compare = pd.DataFrame(data = {
    "model": ["logistic regression", "Random Forest"],
    "overall missclassification rate": [0.115282,
                                       rf_misclass_rate]
}, columns = ["model", "overall missclassification rate"])

misclass_compare
```

```
Out[13]:
```

	model	overall missclassification rate
0	logistic regression	0.115282
1	Random Forest	0.121783

```
In [14]: print("For the random forest, the oob error is " + str(1 - rf.oob_score_) + "
For the random forest, the oob error is 0.12502063319137702, which is very cl
ose to the missclassification error.
```

a2)

```
In [15]: # Logistic Regression
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(C = 100000) # basically no penalization
log_reg.fit(X_train,y_train)

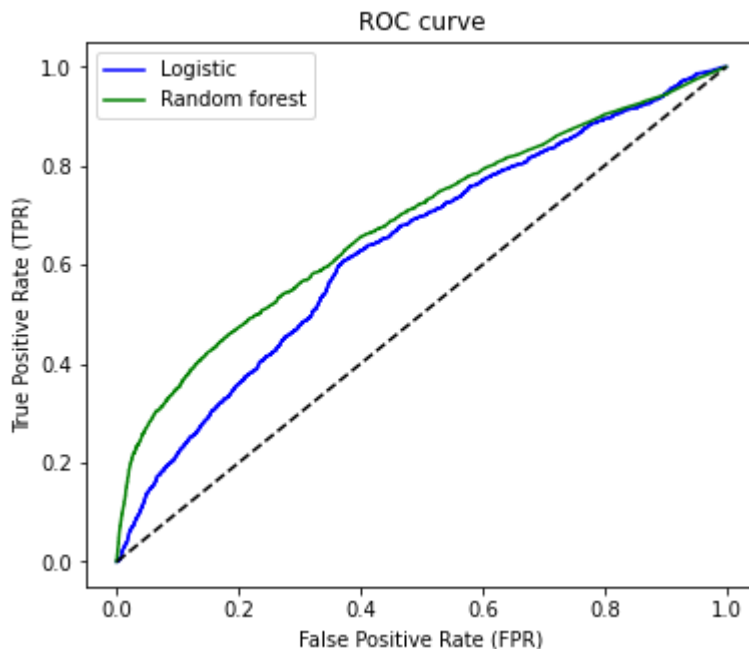
test_prob = log_reg.predict_proba(X_test)[:,-1]
fpr, tpr, _ = roc_curve(y_test, test_prob)
```

/home/maxg/.pyenv/versions/3.9.0/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:762: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
 Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
 sion
 n_iter_i = _check_optimize_result(

```
In [16]: # Random forest
test_prob_rf = rf.predict_proba(X_test)[:,-1]
fpr_rf, tpr_rf, _ = roc_curve(y_test, test_prob_rf)
```

```
In [17]: # plot
fig, ax = plt.subplots(figsize = (6,5))
ax.plot(fpr, tpr, "b-", label = "Logistic")
ax.plot(fpr_rf, tpr_rf, "g-", label = "Random forest")
ax.plot([0,1],[0,1],"k--")
ax.set(xlabel = "False Positive Rate (FPR)",
      ylabel = "True Positive Rate (TPR)",
      title = "ROC curve")
ax.legend();
```



```
In [18]: auc_logistic = metrics.auc(fpr, tpr)
auc_rf = metrics.auc(fpr_rf, tpr_rf)

print("The AUC of logistic regression: " + str(auc_logistic))
print("The AUC of random forest: " + str(auc_rf))
```

The AUC of logistic regression: 0.6343441023608175
The AUC of random forest: 0.6830655831571528

a3)

```
In [19]: # random forest top 1000
top1000_id_rf = np.argsort(test_prob_rf)[::-1][:1000]
rf_success_in_top_1000 = np.mean(np.array(y_test)[top1000_id_rf])

# Logistic regression top 1000
top1000_id = np.argsort(test_prob)[::-1][:1000]
logistic_success_in_top_1000 = np.mean(np.array(y_test)[top1000_id])
logistic_success_in_top_1000

df_compare = pd.DataFrame(data = {
    "model": ["logistic regression", "Random Forest"],
    "overall missclassification rate": [0.115282,
                                       rf_misclass_rate],
    "success rate in top 1000 highest probabilities":
        [logistic_success_in_top_1000, rf_success_in_top_1000]
}, columns = ["model", "overall missclassification rate",
              "success rate in top 1000 highest probabilities"])

df_compare
```

```
Out[19]:
```

	model	overall missclassification rate	success rate in top 1000 highest probabilities
0	logistic regression	0.115282	0.256
1	Random Forest	0.121783	0.443

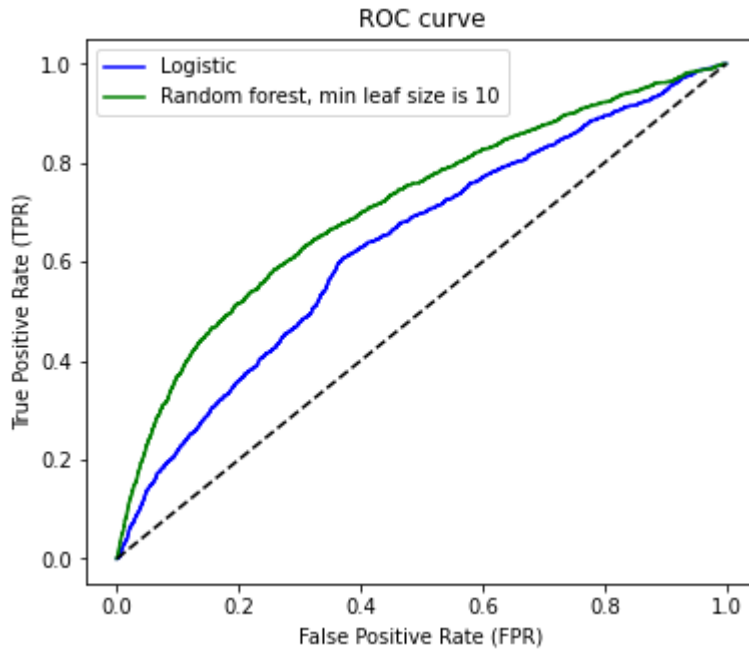
b)

```
In [20]: rf2 = RandomForestClassifier(n_estimators = 500, oob_score = True, min_sample
rf2.fit(X_train, y_train)
```

```
Out[20]: RandomForestClassifier(min_samples_leaf=10, n_estimators=500, oob_score=True)
```

```
In [21]: # ROC
test_prob_rf2 = rf2.predict_proba(X_test)[ :, 1]
fpr_rf2, tpr_rf2, _ = roc_curve(y_test, test_prob_rf2)

fig, ax = plt.subplots(figsize = (6,5))
ax.plot(fpr, tpr, "b-", label = "Logistic")
ax.plot(fpr_rf2, tpr_rf2, "g-", label = "Random forest, min leaf size is 10")
ax.plot([0,1],[0,1], "k--")
ax.set(xlabel = "False Positive Rate (FPR)",
      ylabel = "True Positive Rate (TPR)",
      title = "ROC curve")
ax.legend();
```



```
In [22]: auc_rf2 = metrics.auc(fpr_rf2, tpr_rf2)

print("The AUC of logistic regression: " + str(auc_logistic))
print("The AUC of random forest with min leaf size = 10: " + str(auc_rf2))
```

The AUC of logistic regression: 0.6343441023608175
The AUC of random forest with min leaf size = 10: 0.7103493217054263


```
In [23]: # misclassify
y_test_pred_rf2 = rf2.predict(X_test)
rf2_misclass_rate = np.mean(y_test_pred_rf2 != y_test)

# top 1000
top1000_id_rf2 = np.argsort(test_prob_rf2)[:,-1][:1000]
rf2_success_in_top_1000 = np.mean(np.array(y_test)[top1000_id_rf2])

df_compare = pd.DataFrame(data = {
    "model": ["logistic regression", "Random Forest with min leaf size = 10"]
    "overall missclassification rate": [0.115282,
                                        rf2_misclass_rate],
    "success rate in top 1000 highest probabilities":
        [logistic_success_in_top_1000, rf2_success_in_top_1000]
}, columns = ["model", "overall missclassification rate",
              "success rate in top 1000 highest probabilities"])
df_compare
```

```
Out[23]:
```

	model	overall missclassification rate	success rate in top 1000 highest probabilities
0	logistic regression	0.115282	0.256
1	Random Forest with min leaf size = 10	0.115952	0.380

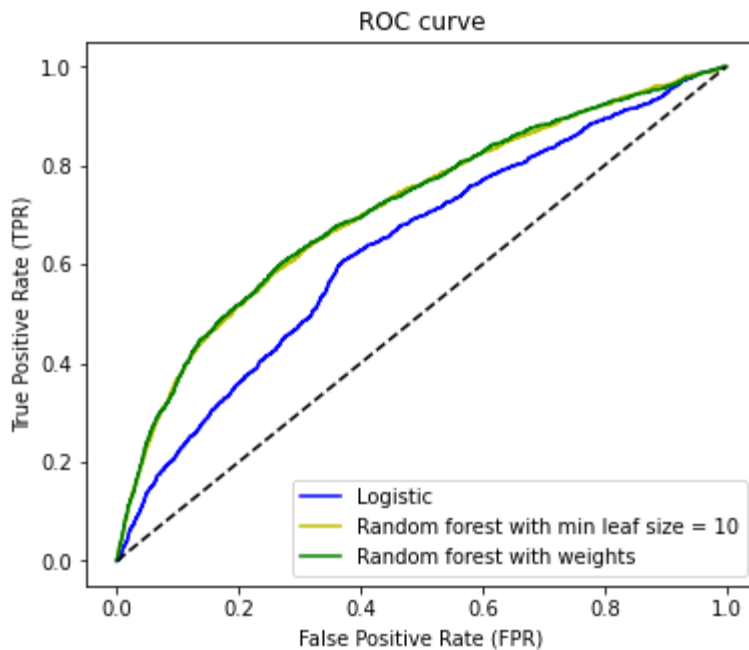
c)

```
In [24]: rf3 = RandomForestClassifier(n_estimators = 500, oob_score = True,
                                     min_samples_leaf = 10,
                                     class_weight = 'balanced')
rf3.fit(X_train, y_train)
```

```
Out[24]: RandomForestClassifier(class_weight='balanced', min_samples_leaf=10,
                                n_estimators=500, oob_score=True)
```

```
In [25]: # ROC
test_prob_rf3 = rf3.predict_proba(X_test)[: , 1]
fpr_rf3, tpr_rf3, _ = roc_curve(y_test, test_prob_rf3)

fig, ax = plt.subplots(figsize = (6,5))
ax.plot(fpr, tpr, "b-", label = "Logistic")
ax.plot(fpr_rf2, tpr_rf2, "y-", label = "Random forest with min leaf size = 10")
ax.plot(fpr_rf3, tpr_rf3, "g-", label = "Random forest with weights")
ax.plot([0,1],[0,1], "k--")
ax.set(xlabel = "False Positive Rate (FPR)",
       ylabel = "True Positive Rate (TPR)",
       title = "ROC curve")
ax.legend();
```



```
In [26]: # AUC
auc_rf3 = metrics.auc(fpr_rf3, tpr_rf3)

print("The AUC of logistic regression: " + str(auc_logistic))
print("The AUC of random forest with min leaf size = 10: " + str(auc_rf2))
print("The AUC of random forest with weights: " + str(auc_rf3))
```

The AUC of logistic regression: 0.6343441023608175
The AUC of random forest with min leaf size = 10: 0.7103493217054263
The AUC of random forest with weights: 0.7120519732205779

The classification rate of the weighted random forest is worse than the unweighted one. However, the weighted random forest has larger AUC. The weighted random forest has higher misclassification rate because it assigns the positive label to more input points.

```
In [27]: # misclassification rate
y_test_pred_rf3 = rf3.predict(X_test)
rf3_misclass_rate = np.mean(y_test_pred_rf3 != y_test)

# top 1000
top1000_id_rf3 = np.argsort(test_prob_rf3)[::-1][:1000]
rf3_success_in_top_1000 = np.mean(np.array(y_test)[top1000_id_rf3])

df_compare = pd.DataFrame(data = {
    "model": ["logistic regression", "Random Forest with weights"],
    "overall misclassification rate": [0.115282,
                                     rf3_misclass_rate],
    "success rate in top 1000 highest probabilities":
        [logistic_success_in_top_1000, rf3_success_in_top_1000]
}, columns = ["model", "overall misclassification rate",
              "success rate in top 1000 highest probabilities"])

df_compare
```

```
Out[27]:
```

model	overall misclassification rate	success rate in top 1000 highest probabilities
logistic regression	0.115282	0.7103493217054263
Random Forest with weights	0.115282	0.7120519732205779

	model	overall missclassification rate	success rate in top 1000 highest probabilities
0	logistic regression	0.115282	0.256

```
In [28]: # misclassification rate
y_test_pred_rf3 = rf3.predict(X_test)
rf3_misclass_rate = np.mean(y_test_pred_rf3 != y_test)

# top 1000
top1000_id_rf3 = np.argsort(test_prob_rf3)[::-1][:1000]
rf3_success_in_top_1000 = np.mean(np.array(y_test)[top1000_id_rf3])

df_compare = pd.DataFrame(data = {
    "model": ["logistic regression", "Random Forest with weights"],
    "overall missclassification rate": [0.115282,
                                         rf3_misclass_rate],
    "success rate in top 1000 highest probabilities":
        [logistic_success_in_top_1000, rf3_success_in_top_1000]
}, columns = ["model", "overall missclassification rate",
              "success rate in top 1000 highest probabilities"])
df_compare
```

```
Out[28]:
```

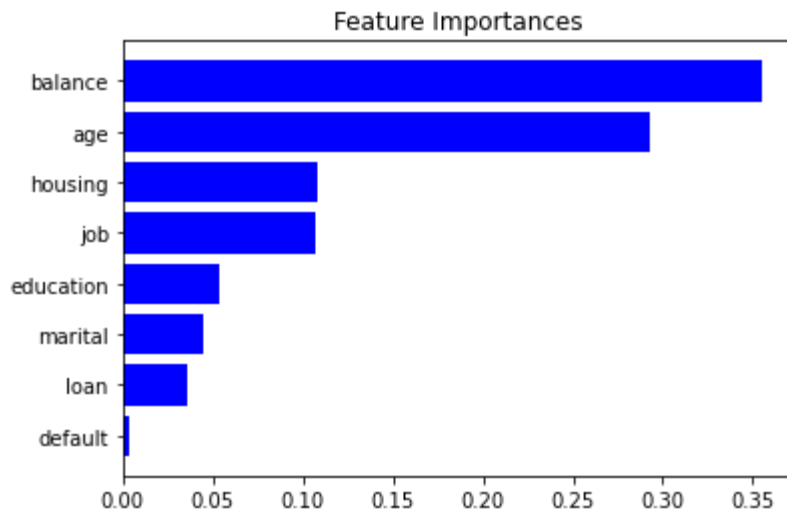
	model	overall missclassification rate	success rate in top 1000 highest probabilities
0	logistic regression	0.115282	0.256
1	Random Forest with weights	0.218499	0.392

Problem 3

Variable importance

```
In [29]: X_features = list(X_train.columns)
rf_importances = rf3.feature_importances_
indices = np.argsort(rf_importances)

fig, ax = plt.subplots()
plt.xticks(range(len(indices)), [X_features[i] for i in indices])
ax.barh(range(len(indices)), rf_importances[indices], color='b', align='center')
ax.set(title = 'Feature Importances')
plt.show()
```



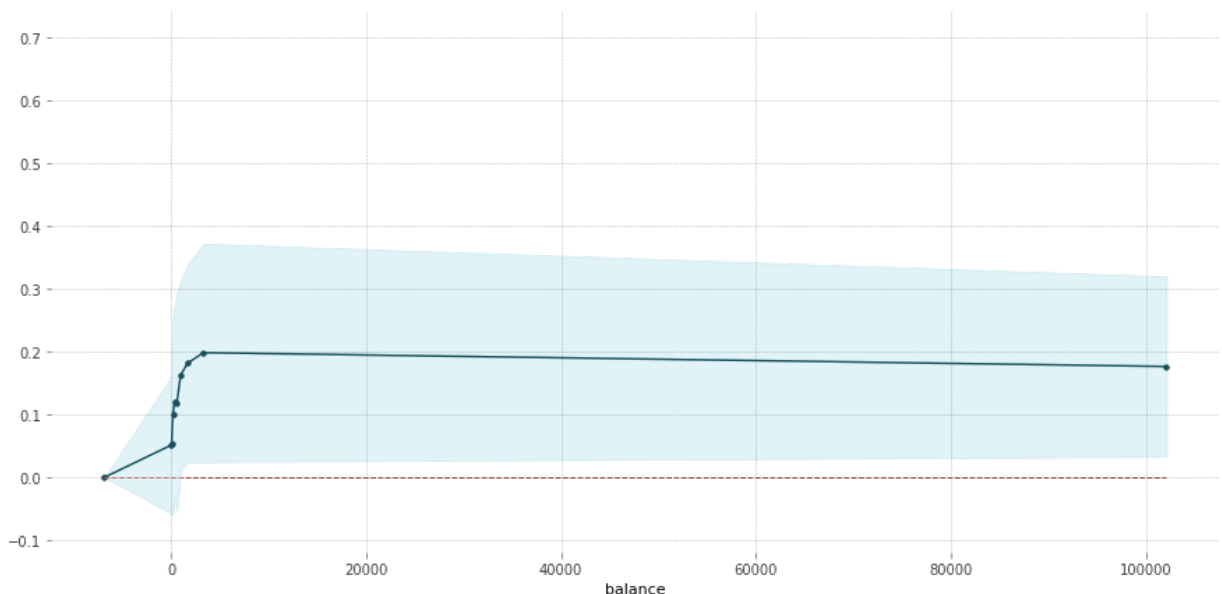
The two most important variables are balance and age.

Partial dependence plots for the two most important variables

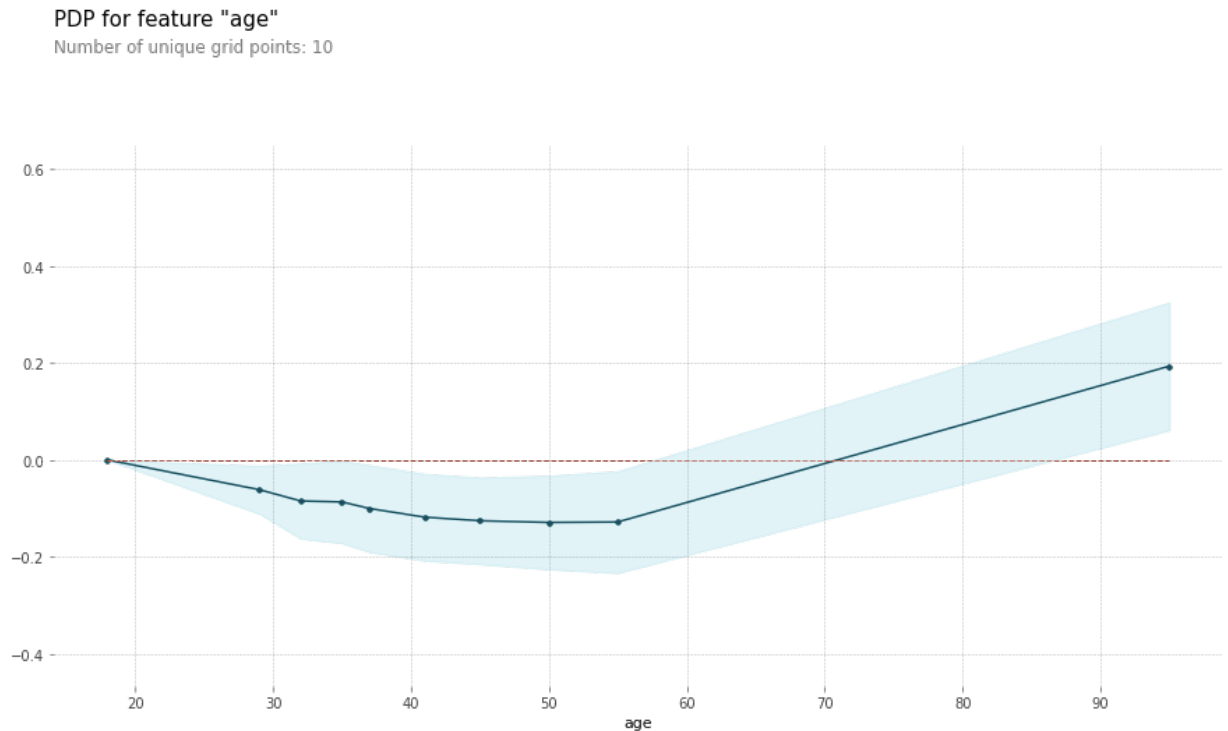
```
In [30]: from pdpbox import pdp, get_dataset, info_plots
pdp_obj = pdp.pdp_isolate(model = rf3, dataset = X_train, model_features = X_
                    feature = 'balance')
fig, axes = pdp.pdp_plot(pdp_obj, 'balance')
```

findfont: Font family ['Arial'] not found. Falling back to DejaVu Sans.
 findfont: Font family ['Arial'] not found. Falling back to DejaVu Sans.
 findfont: Font family ['Arial'] not found. Falling back to DejaVu Sans.
 findfont: Font family ['Arial'] not found. Falling back to DejaVu Sans.

PDP for feature "balance"
 Number of unique grid points: 10



```
In [31]: pdp_obj = pdp.pdp_isolate(model = rf3, dataset = X_train, model_features = X_
        feature = 'age')
fig, axes = pdp.pdp_plot(pdp_obj, 'age')
```



Problem 4

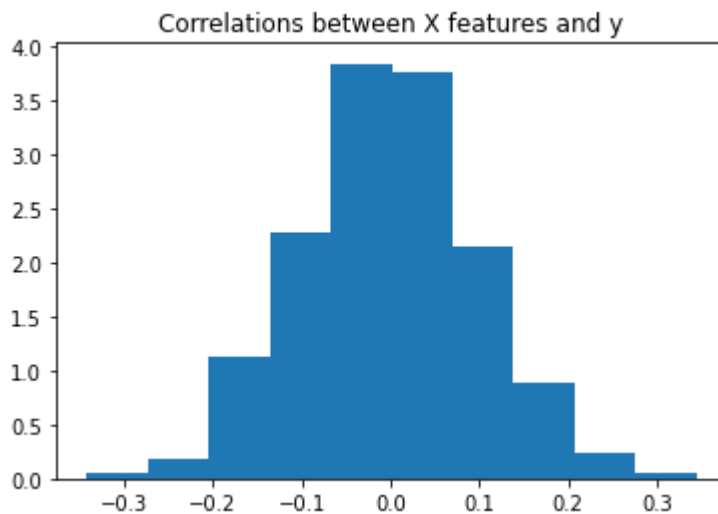
(a)

```
In [32]: def get_data(n):
        p=1000
        X = scipy.stats.norm().rvs(n*p).reshape((n,p))
        Y = scipy.stats.norm().rvs(n)
        return(X,Y)
```

```
In [33]: # def generate_data(n=100):
        #     X, y = get_data(n)
        #     return X, y
```

```
In [34]: #X, y = generate_data(n=100)
        n=100
        X, y = get_data(n=n)
```

```
In [35]: X, y = get_data(n=100)
corrs = np.apply_along_axis(pearsonr, 0, X, y)[0,:]
plt.hist(corrs, density=True)
plt.title("Correlations between X features and y")
plt.show()
```



(b)

```
In [36]: idx_good = np.abs(corrs) > 0.2
X_good = X[:, idx_good]
```

(c)

```
In [37]: X_train = X_good[ : int(0.8*n) ]
y_train = y[ : int(0.8*n)]

X_test = X_good[int(0.8*n) : ]
y_test = y[int(0.8*n) : ]

lasso = LassoCV(cv=10).fit(X_train, y_train)
```

```
In [38]: all_CV_MSEs = np.mean(lasso.mse_path_, axis=1)
CV_MSE = all_CV_MSEs[np.argmin(all_CV_MSEs)]
print("CV estimate for MSE:", round(CV_MSE, 3))
```

CV estimate for MSE: 0.464

```
In [39]: y_pred = lasso.predict(X_test)
test_MSE = mse(y_test, y_pred)
print("Test MSE:", round(test_MSE, 3))
```

Test MSE: 0.555

(d)

```
In [40]: X_new, y_new = get_data(n=100)
X_good_new = X_new[:, idx_good]
```

```
In [41]: metrics.mean_squared_error(y_new, lasso.predict(X_good_new))
```

```
Out[41]: 1.3928177075497756
```

In part c, we saw that the out of sample error (on test) was similar to the CV estimate for MSE (on train). However, when we generate a fresh new dataset, but use the old CV-chosen model from part c, the new MSE is much higher.

The key is that in part b, we screened for X variables with at least 0.2 correlation with y to form X_good, which caused the MSEs to be artificially lower in part c. But these are all spurious correlations that do not carry over to X_new.